LLM-Optimised Data Querying

Step 1: Create a Data Warehouse

Create a database using SQLite to store structured data.

LLM-Optimised Data Querying.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[17] 0s

```
import sqlite3
```

[18] 0s

```
conn = sqlite3.connect("warehouse.db")
cursor = conn.cursor()
```

Step 2: Design Tables

Create tables such as Customers and Orders to organize data.

[19] 0s

```
cursor.execute("""
CREATE TABLE customers (
    customer_id INT,
    name TEXT,
    region TEXT
);
""")

cursor.execute("""
CREATE TABLE orders (
    order_id INT,
    customer_id INT,
    sales_amount FLOAT,
    order_date TEXT
);
""")

conn.commit()
print("Tables created")
```

```
Tables created
```

Step 3: Insert Data

Add sample data into the tables for testing and analysis.

[20]
0s

```python
cursor.executemany("""
INSERT INTO customers VALUES (?, ?, ?)
""", [
    (1, 'Arjun', 'South'),
    (2, 'Meena', 'North'),
    (3, 'Rahul', 'East'),
    (4, 'Sara', 'West')
])

cursor.executemany("""
INSERT INTO orders VALUES (?, ?, ?, ?)
""", [
    (101, 1, 5000, '2025-01-01'),
    (102, 2, 7000, '2025-01-02'),
    (103, 3, 3000, '2025-01-03'),
    (104, 1, 2000, '2025-01-04')
])

conn.commit()
print("Data inserted")
```

```
Data inserted
```

Step 4: Verify Database

Run basic SQL queries to ensure data is stored correctly.

[21]
0s

```python
cursor.execute("SELECT * FROM customers")
print(cursor.fetchall())
```

```
[(1, 'Arjun', 'South'), (2, 'Meena', 'North'), (3, 'Rahul', 'East'), (4, 'Sara', 'West')]
```

Step 5: Implement LLM Logic

Create a module that converts natural language queries into SQL queries (simulated LLM).

LLM Logic (Mock AI)

[22]
0s

```python
def generate_sql(query):
    query = query.lower()

    if "all customers" in query:
        return "SELECT * FROM customers"

    elif "sales" in query and "total" in query:
        return "SELECT SUM(sales_amount) FROM orders"

    elif "region" in query:
        return "SELECT region, COUNT(*) FROM customers GROUP BY region"

    elif "orders" in query:
        return "SELECT * FROM orders"

    else:
        return "SELECT * FROM customers"
```

Step 6: Integrate System

Connect user input, LLM logic, and database execution into a single workflow.

[23] ✓ 0s

```
def run_query(sql):
    cursor.execute(sql)
    return cursor.fetchall()
```

Step 8: Display Results

Show query results

[24] ✓ 9s

```
user_input = input("Ask your question: ")

sql = generate_sql(user_input)
print("Generated SQL:", sql)

result = run_query(sql)
print("Result:", result)
```

```
Ask your question: show all customers
Generated SQL: SELECT * FROM customers
Result: [(1, 'Arjun', 'South'), (2, 'Meena', 'North'), (3, 'Rahul', 'East'), (4, 'Sara', 'West')]
```

[25] ✓ 6s

```
user_input = input("Ask your question: ")

sql = generate_sql(user_input)
print("Generated SQL:", sql)

result = run_query(sql)
print("Result:", result)
```

```
Ask your question: total sales
Generated SQL: SELECT SUM(sales_amount) FROM orders
Result: [(17000.0,)]
```

[26] ✓ 7s

```
user_input = input("Ask your question: ")

sql = generate_sql(user_input)
print("Generated SQL:", sql)

result = run_query(sql)
print("Result:", result)
```

```
Ask your question: customers by region
Generated SQL: SELECT region, COUNT(*) FROM customers GROUP BY region
Result: [('East', 1), ('North', 1), ('South', 1), ('West', 1)]
```

[27] ✓ 9s

```
user_input = input("Ask your question: ")

sql = generate_sql(user_input)
print("Generated SQL:", sql)

result = run_query(sql)
print("Result:", result)
```

```
Ask your question: show orders
Generated SQL: SELECT * FROM orders
Result: [(101, 1, 5000.0, '2025-01-01'), (102, 2, 7000.0, '2025-01-02'), (103, 3, 3000.0, '2025-01-
```